

SYSTEM DESIGN REPORT

Multi-Accused Cross-Account Investigation Engine

Problem Statement 1 — Automated Bank Statement Analysis System

Team	Survey Corps
Document Version	Version 2.0 — June 2026 (supersedes Version 1.0)
Stage	Stage 1 — Requirements, System Design & Analysis Phase Specification
Prepared During	Mentoring Phase (05–25 June 2026), ahead of the 24-hour hackathon on 4–5 July 2026
Architecture	Validation-Arbitrated Tiered Hybrid Extraction Engine

This document supersedes Version 1.0. Section 4.2 (Extraction Module) has been completely rewritten to reflect the Validation-Arbitrated Tiered Hybrid architecture implemented during the mentoring phase. All other sections are carried forward unchanged from v1.0.

Table of Contents

1. Purpose of This Revision
2. What Changed Since Version 1.0
3. Design Goal & Guiding Principles
4. Core Philosophy — Validation-Arbitrated Tiered Hybrid
 - 4.1 The Three Governing Principles
5. The Anti-Overfitting Law
6. Extraction Architecture — The Five Tiers
 - 6.1 Tier Diagram
 - 6.2 Tier Descriptions
7. The Validator — Heart of the System
 - 7.1 The Four Checks
 - 7.2 Four Tricky Cases
 - 7.3 Worked Validation Examples
8. The Deterministic Schema-Driven Engine
9. The Escalation Ladder
10. The LLM Interface — Provider-Independent
11. Worked End-to-End Example
 - 11.1 Three-File Scenario
 - 11.2 Sample Output Table
12. Output Schema & Storage
13. Privacy & Security
14. Cost, Performance & Local-Model Path
15. Testing & Accuracy Proof
16. Unchanged Downstream Phases
 - 16.1 Analysis Phase (25 Detection Cases)
 - 16.2 Report Generation
 - 16.3 RAG Investigation Chatbot
17. Requirements
18. System Architecture Overview
19. Technology Stack
20. Data Storage & Data Flow
21. Security & Privacy Considerations
22. Scalability & Deployment
23. Implementation Roadmap
24. Team Structure & Roles
25. Conclusion

1. Purpose of This Revision

This document is Version 2.0 of the System Design Report for the Multi-Accused Cross-Account Investigation Engine, produced by Team Survey Corps for the CIDECODE Hackathon 2026. It supersedes Version 1.0 (June 2026) in its entirety.

Version 1.0 was written before the extraction module was implemented. During implementation, the extraction pipeline evolved significantly from the original design and developed several problems:

- Two contradictory parsing strategies coexisted (LLM-per-row and pandas-bulk), causing silent data loss.
- Token usage scaled with row count rather than document count, making large datasets prohibitively expensive.
- A column-map applied to every row without per-row validation created an overfitting risk to specific bank formats.
- A silent data-loss bug dropped rows that failed validation without surfacing them for manual review.

The team resolved these issues by agreeing on a corrected architecture: the **Validation-Arbitrated Tiered Hybrid**. This document rewrites Section 4.2 (Extraction Module) to describe that architecture. All other sections are carried forward faithfully from Version 1.0.

2. What Changed Since Version 1.0

The table below summarises every material difference between v1.0 and v2.0.

Area	Version 1.0 (outdated)	Version 2.0 (this document)
Extraction philosophy	LLM identifies columns once; pandas parses every row uniformly.	Validation-Arbitrated Tiered Hybrid: cheap deterministic parse first, validator decides escalation.
LLM role in extraction	LLM called once per document for column structure; result applied blindly to all rows.	LLM called once per document for schema discovery (Tier 4), and only on failing rows for repair (Tier 5). Never writes per-row parsers.
Token cost	Scales with number of documents (one call each). Rows processed by pandas.	Identical: tokens scale with documents, not rows. Tier 5 adds calls only for genuinely failing rows.
Validation	Component 5: three checks (date, balance arithmetic, debit/credit exclusivity). Failures flagged.	Dedicated Validator module with four checks + four tricky-case handlers. Acts as the system referee — decides escalation, not just flagging.
Overfitting risk	Column map could be hard-coded or bank-specific without explicit guard.	Anti-Overfitting Law: no bank-name branching; schema parameters from LLM discovery, not hard-coded; correctness proven on a blind set.
Parsing engine	Implicit: pandas reads rows using the LLM-supplied column map.	One deterministic engine parameterised by a schema object. Same engine for every bank.
Escalation	Not formalised. Failures go to flagged list.	Named escalation ladder with three config thresholds (ACCEPT_RECONCILE_RATE, MIN_COMPLETENESS_RATIO, BALANCE_TOLERANCE).

Area	Version 1.0 (outdated)	Version 2.0 (this document)
LLM provider	LLM API cited as Groq / Claude / GPT-4 interchangeably.	Provider-independent interface: four functions behind one module. Swap local model by changing one file.
Output schema	Date, Narration, Debit, Credit, Balance, Account_ID, Bank_Name.	Adds IFSC_Code and Time. Plus clean CSV, flagged CSV, and metadata receipt as separate artefacts.
Unchanged	—	All analysis cases (Sec 5), reporting (Sec 6), RAG chatbot (Sec 7), technology stack (Sec 8), storage (Sec 9), security (Sec 10), scalability (Sec 11), roadmap (Sec 12).

3. Design Goal & Guiding Principles

The extraction module has one job: take any combination of PDF, Excel, image, CSV or DOCX files and produce one unified transaction table, every time, regardless of which bank issued the statement. The design goal is to do this correctly, cheaply, and in a way that is legally defensible.

Correctness over convenience	A row that cannot be parsed correctly must be flagged, not guessed. Every clean row must pass balance arithmetic before it enters the analysis phase.
Determinism where possible	Cheap, rule-based parsing runs first. The LLM is called only when deterministic methods provably fail. This keeps the system reproducible and cost-efficient.
Bank-agnostic universality	One engine, one set of parameters, any bank. No if-HDFC-then-X branches. The schema is discovered at runtime, not hard-coded at design time.
Transparency uncertainty	Flagged rows are surfaced to the investigator, never silently dropped. The metadata receipt records what was accepted, what was flagged, and why.
Provider independence	The LLM interface is a thin module with four functions. Swapping from Groq to a local model (Ollama, LM Studio) requires changing exactly one file.

4. Core Philosophy — Validation-Arbitrated Tiered Hybrid

The Validation-Arbitrated Tiered Hybrid is the name for the extraction architecture agreed on by the team after implementation. It is called 'tiered' because processing escalates through tiers of increasing cost only when cheaper tiers fail. It is called 'validation-arbitrated' because a deterministic validator — not a human, not the LLM — decides at each tier whether the output is good enough to proceed or must escalate.

4.1 The Three Governing Principles

The entire architecture rests on three principles that cannot be violated without breaking the design's integrity:

Principle 1 — The LLM fills parameters; humans write the engine	The LLM is permitted to return a schema object (column positions, date format, currency symbol) and to suggest repairs for specific failing rows. It is never permitted to write a parser, a regex, or any code that runs against the data. All row-level parsing logic is written by the team and lives in the deterministic engine.
--	---

Principle 2
— The
balance
equation is
the
universal
referee

For every parsed row the $\text{check balance_prev} + \text{credit} - \text{debit} = \text{balance_curr}$ must hold within `BALANCE_TOLERANCE` (default: Rs 1.00). This equation is bank-agnostic — it works for SBI, HDFC, Axis, ICICI, and any bank not yet seen. No heuristic, no LLM judgement, no human eye replaces this check.

Principle 3
— Spend
intelligence
only where
cheap
methods
provably
fail

Tier 2 (cheap deterministic parse) runs first. The validator checks its output. Only rows that fail the validator escalate to Tier 4 (LLM schema discovery) and Tier 5 (LLM row repair). A document where every row passes Tier 2 incurs zero LLM cost beyond the initial metadata call.

5. The Anti-Overfitting Law

A system that works perfectly on HDFC statements but silently misparses Canara Bank statements is worse than useless in a forensic context — it produces authoritative-looking errors. The Anti-Overfitting Law is a set of rules that prevent this:

**No
bank-name
branching**

The extraction code must not contain any conditional logic of the form 'if bank == HDFC then ...' or 'if bank == SBI then ...'. Every bank's statement is processed by exactly the same code path. Differences in layout are handled by the schema discovery step (Tier 4), which returns a bank-agnostic schema object.

**Parameters
from
schema,
not
hard-coded**

Column indices, date formats, currency symbols, and header row numbers are discovered at runtime by the LLM (Tier 4) and stored in a schema object. They are never constants in the source code.

**Correctness
proven on
a blind set**

Before the hackathon demo, the extraction pipeline must be tested against a blind set of statements from banks that were NOT used during development. A pass rate below `ACCEPT_RECONCILE_RATE` on any blind-set document is a failing grade and triggers a design review, not a special-case patch.

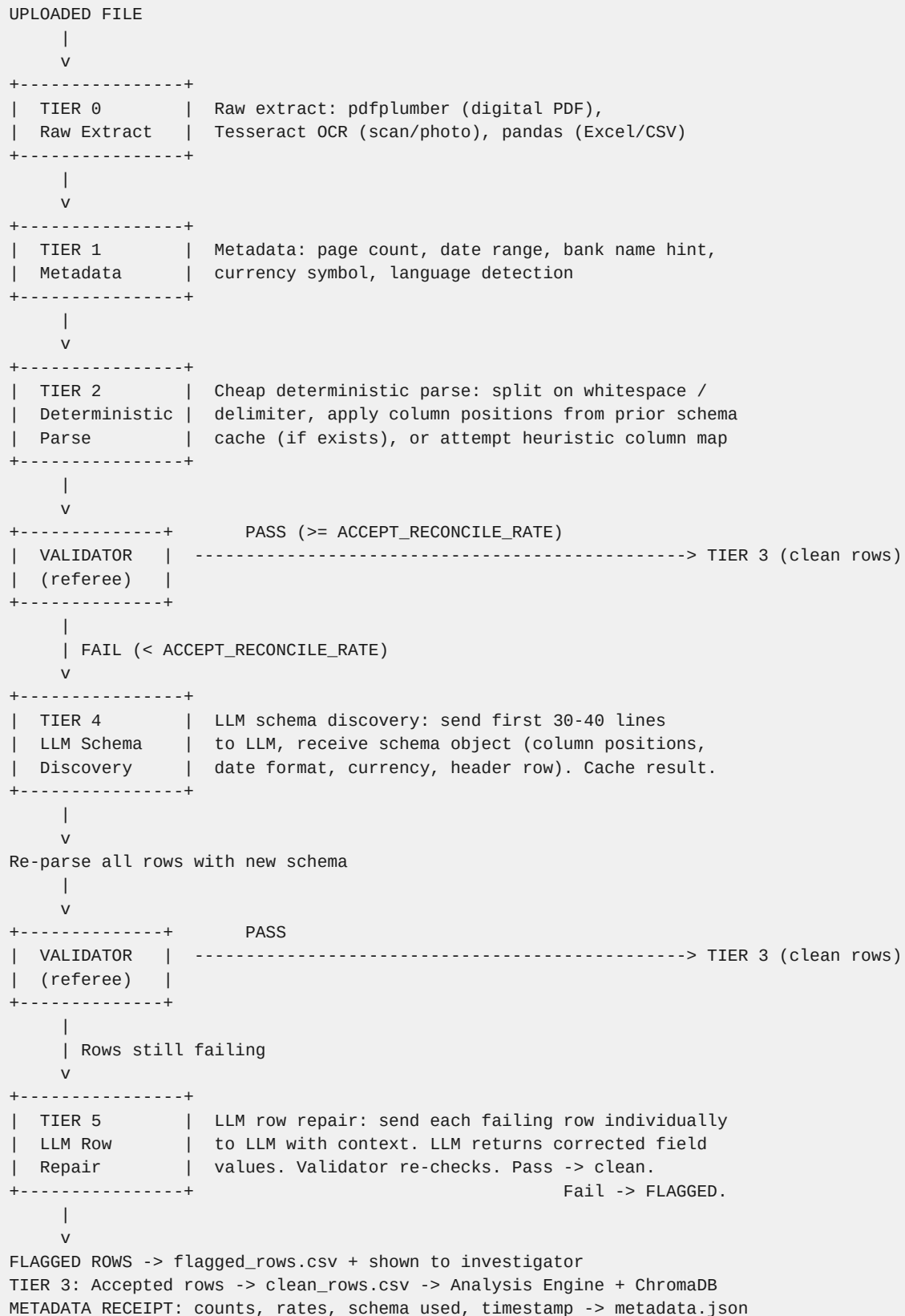
**No special-
case
patches**

If a document fails, the fix must be a general improvement to the engine or the schema-discovery prompt — never a bank-specific workaround. A workaround is itself a violation of the Anti-Overfitting Law.

6. Extraction Architecture — The Five Tiers

The extraction pipeline is organised into six tiers (Tier 0 through Tier 5). Each tier hands its output to the Validator. The Validator decides whether to accept the output, escalate to the next tier, or flag the row for manual review. Tiers 4 and 5 involve the LLM; Tiers 0–3 are fully deterministic.

6.1 Tier Diagram



6.2 Tier Descriptions

Tier 0 — Raw Extract

The file-type router sends each uploaded file to the correct extractor. pdfplumber handles digital PDFs (text embedded in the file). Tesseract OCR handles scanned PDFs and phone photographs. pandas reads Excel and CSV files directly. The output of Tier 0 is raw text (for PDF/image) or a raw dataframe (for Excel/CSV). No parsing decisions are made here.

Tier 1 — Metadata

A lightweight pass over the raw text identifies: approximate page count, the date range of visible transactions, a candidate bank-name string (from the document header), the currency symbol in use, and whether the text appears to be in a recognised language. This metadata is stored in the metadata receipt and used to seed the LLM schema-discovery prompt in Tier 4 if needed.

Tier 2 — Cheap Deterministic Parse

The deterministic engine attempts to parse every row using either a cached schema from a previous document of the same format, or a heuristic column map (date-like token, numeric tokens, remainder as narration). This is the fastest and cheapest path. For well-formatted digital PDFs and Excel files, Tier 2 typically achieves a pass rate above ACCEPT_RECONCILE_RATE and the pipeline terminates here.

Tier 3 — Accepted Rows

Rows that pass the Validator's four checks (see Section 7) are written to clean_rows.csv and loaded into the in-memory pandas dataframe used by the analysis engine. They are also simultaneously written to ChromaDB as searchable vector embeddings for the RAG chatbot. Tier 3 is a destination, not a processing step.

Tier 4 — LLM Schema Discovery

Triggered when Tier 2's pass rate falls below ACCEPT_RECONCILE_RATE. The first 30–40 lines of the document are sent to the LLM with the metadata from Tier 1. The LLM returns a schema object: column indices (or names), date format string, credit/debit indicator type (separate columns vs. sign-coded single column), and header row number. The schema object is cached locally so identical document formats do not incur repeated API calls. All rows are then re-parsed by the deterministic engine using the new schema. The Validator re-checks the output.

Tier 5 — LLM Row Repair

If individual rows still fail the Validator after Tier 4, each failing row is sent to the LLM individually with the surrounding two rows as context. The LLM returns corrected field values (not code, not a parser — just the values for Date, Narration, Debit, Credit, Balance). The Validator re-checks each repaired row. Rows that pass go to Tier 3. Rows that still fail are written to flagged_rows.csv and shown to the investigator with a warning. LLM calls in Tier 5 scale with the number of failing rows, but because most failures come from OCR noise on a small subset of rows, the call count remains small in practice.

7. The Validator — Heart of the System

The Validator is the only component that can promote a row to Tier 3 (clean) or escalate it to the next tier. It is called after every parsing attempt at every tier. It applies four checks and handles four categories of tricky cases that naive implementations get wrong.

7.1 The Four Checks

Check	Rule	Failure Meaning
Check 1 — Balance Reconciliation	$\text{balance_prev} + \text{credit} - \text{debit} = \text{balance_curr}$	Within BALANCE_TOLERANCE (Rs 1.00 default). Failure means the row was misaligned or a value was misread.
Check 2 — Debit/Credit Exclusivity	NOT (debit > 0 AND credit > 0)	Exactly one of debit or credit should be positive per row. Both positive means column misalignment.
Check 3 — Date Validity	date parses as a real calendar date	A value like '90000' or '3/13/2024' (month 13) in the date column indicates a row shift or OCR misread.
Check 4 — Completeness	narration is not blank AND at least one of debit/credit is non-zero	A row with no narration and no amount is likely a header repetition or OCR garbage.

7.2 Four Tricky Cases

Naive implementations fail on these four situations. The Validator handles them explicitly:

Newest-first statements Some banks (e.g., certain credit card statements) list transactions in reverse chronological order. A naive ascending balance check flags every row. The Validator detects reverse order from Tier 1 metadata and applies the balance equation in the correct direction: $\text{balance_curr} + \text{debit} - \text{credit} = \text{balance_prev}$.

No opening balance Some statement extracts omit the opening balance row. Without it, balance_prev for the first transaction is unknown and the reconciliation check cannot run. The Validator treats the first row's balance as a trusted anchor, marks it with a special flag in the metadata receipt, and begins reconciliation from the second row.

No balance column Some formats (certain UPI export CSVs) do not include a running balance column at all. Check 1 cannot be applied. The Validator detects the missing column from the schema object, disables Check 1 for that document, and relies on Checks 2, 3 and 4 only. The metadata receipt records this limitation explicitly.

Wrapped narrations Long narration text sometimes wraps to the next physical line, causing the parser to treat the continuation as a new row with missing numeric fields. The Validator detects this pattern (missing debit, credit, and balance on a row where the narration is non-empty) and merges the continuation line into the previous row's narration field rather than flagging it.

7.3 Worked Validation Examples

Example A — Clean row (passes all four checks)

```
Row: Date=15/03/2024 Narration="UPI/PhonePe/RAMESH" Debit=5000 Credit=0 Balance=12000
Previous balance from prior row: 17000
Check 1 (balance reconciliation):  $17000 - 5000 + 0 = 12000$  PASS
Check 2 (debit/credit exclusivity): debit=5000, credit=0 -> only one populated PASS
Check 3 (date validity): 15/03/2024 parses as 15 March 2024 PASS
Check 4 (completeness): narration non-empty, debit non-zero PASS
Result: ROW ACCEPTED -> Tier 3 (clean_rows.csv)
```

Example B — Misaligned row (fails Check 1, escalates)

```
Row: Date=16/03/2024 Narration="NEFT/HDFC/SUNITA" Debit=0 Credit=8000 Balance=18000
Previous balance from prior row: 12000
Check 1 (balance reconciliation): 12000 + 8000 - 0 = 20000 BUT row says 18000 FAIL
Delta = Rs 2000 (exceeds BALANCE_TOLERANCE of Rs 1.00)
Result: ROW ESCALATED -> sent to Tier 5 (LLM row repair)
LLM repair: suggests Credit=6000 (narration references partial payment of Rs 6000)
Re-check: 12000 + 6000 = 18000 PASS
Result: REPAIRED ROW ACCEPTED -> Tier 3
```

Example C — OCR garbage row (fails multiple checks, flagged)

```
Row: Date="90000" Narration="" Debit=0 Credit=0 Balance=0
(This is an OCR misread: two rows merged into one garbage line)
Check 3 (date validity): "90000" does not parse as a date FAIL
Check 4 (completeness): narration blank AND both amounts zero FAIL
Result: ROW FLAGGED -> flagged_rows.csv, shown to investigator with warning:
"Row 47: Date field contains '90000' – possible OCR merge. Manual review required."
```

8. The Deterministic Schema-Driven Engine

One parsing engine handles every bank statement. It accepts a schema object and a raw text block (or dataframe) and applies the schema to produce rows in the standard output format. The schema object is the only thing that varies between banks; the engine itself does not.

Schema Object Structure

```
{
  "date_col":      0,          // column index (0-based) or column name
  "narration_col": 1,
  "debit_col":     2,
  "credit_col":    3,
  "balance_col":   4,
  "time_col":      null,      // null if no separate time column
  "ifsc_col":      null,
  "date_format":   "%d/%m/%Y", // strftime format string
  "currency_symbol": "Rs",
  "header_rows":   1,        // number of header rows to skip
  "reverse_order": false,    // true if newest-first
  "balance_present": true,    // false if no balance column
  "sign_coded":    false     // true if single Amt col with +/- sign
}
```

The engine performs these steps in order for every row: (1) extract raw token at each column index, (2) strip currency symbols and commas from numeric fields, (3) parse the date string using `date_format`, (4) apply sign-coded logic if `sign_coded` is true, (5) attach `Account_ID`, `Bank_Name` and `IFSC_Code` from the Tier 1 metadata, (6) pass the completed row to the Validator. No step in this list contains any bank-specific logic.

The schema object is cached in a local JSON file keyed by a hash of the document's first 30 lines. On subsequent runs with the same document format, the Tier 4 LLM call is skipped entirely and the cached schema is used directly.

9. The Escalation Ladder

The escalation ladder formalises the conditions under which processing moves from one tier to the next. Three named configuration constants govern all decisions. These constants can be adjusted in a single config file; no code changes are required.

Configuration Constants

```
# config/extraction.py
ACCEPT_RECONCILE_RATE = 0.98 # Min fraction of rows passing balance check
                                # at Tier 2 before Tier 4 is triggered.
                                # 0.98 = 98 out of 100 rows must reconcile.
MIN_COMPLETENESS_RATIO = 0.90 # Min fraction of rows with non-blank narration
                                # AND non-zero amount before the document is
                                # considered parseable at all.
                                # Below this -> entire document flagged.
BALANCE_TOLERANCE = 1.0       # Maximum absolute difference (in document
                                # currency units) between computed and stated
                                # balance before a row is considered failing.
                                # Default: Rs 1.00 (handles rounding artefacts).
```

Decision Table

Condition	Action
Tier 2 reconcile rate $\geq$ ACCEPT_RECONCILE_RATE AND completeness $\geq$ MIN_COMPLETENESS_RATIO	Accept all passing rows to Tier 3. No LLM call.
Tier 2 reconcile rate $<$ ACCEPT_RECONCILE_RATE AND completeness $\geq$ MIN_COMPLETENESS_RATIO	Trigger Tier 4 (LLM schema discovery). Re-parse. Re-validate.
After Tier 4, reconcile rate $\geq$ ACCEPT_RECONCILE_RATE	Accept all passing rows to Tier 3. Failing rows go to Tier 5.
After Tier 4, individual rows still failing	Trigger Tier 5 (LLM row repair) for each failing row. Validator re-checks.
After Tier 5, row still failing	Row written to flagged_rows.csv. Investigator notified.
Completeness $<$ MIN_COMPLETENESS_RATIO at any tier	Entire document flagged. Not partially accepted. Investigator notified with diagnosis.

10. The LLM Interface — Provider-Independent

All LLM calls in the extraction pipeline (and, by extension, in the analysis and chatbot phases) are routed through a single module: `llm_interface.py`. This module exposes exactly four functions. The rest of the codebase imports only these four functions and has no knowledge of which provider is in use.

```
# extraction/llm_interface.py
def discover_schema(raw_text: str, metadata: dict) -> dict:
    """Tier 4: send first 30-40 lines + metadata. Return schema object."""
def repair_row(raw_row: str, context_rows: list, schema: dict) -> dict:
    """Tier 5: send one failing row + 2 context rows. Return corrected fields."""
def generate_thresholds(case_brief: str) -> dict:
    """Analysis phase: send case brief. Return threshold JSON for 25 cases."""
def explain_finding(finding: dict, case_brief: str) -> dict:
    """Analysis phase: send one finding object. Return explanation + rating + action."""
```

To switch from Groq to a local model (Ollama, LM Studio, vLLM), the team replaces the implementation inside `llm_interface.py` — changing the HTTP endpoint, the authentication header, and the model name string. No other file changes. The four function signatures remain identical. This is what makes the system local-model-ready without any redesign.

Function	Caller	Input tokens (approx.)	Calls per document
discover_schema	Tier 4 escalation	~500–800 (30-40 lines of text)	0 (cache hit) or 1
repair_row	Tier 5 escalation	~150–300 per row	0 to N failing rows
generate_thresholds	Analysis phase (Case 23)	~200–400 (case brief)	1 per session
explain_finding	Analysis phase (LLM explanation layer)	~300–600 per finding	1 per triggered case per account

11. Worked End-to-End Example

The following example shows how three real-world file types flow through the extraction pipeline and land in one unified table. This is the three-file scenario used by the team during development testing.

11.1 Three-File Scenario

File 1 — Digital PDF (SBI statement, 120 rows)	Tier 0: pdfplumber extracts clean text. Tier 1: metadata identifies SBI, date range Jan-Mar 2024, Rs currency. Tier 2: heuristic column map applied. Validator: reconcile rate = 1.00 (all 120 rows pass). No Tier 4 or 5 needed. All 120 rows -> Tier 3 (clean). LLM calls: 0.
File 2 — Excel file (Axis Bank export, 340 rows)	Tier 0: pandas reads directly. Tier 1: column names present ('Date', 'Particulars', 'Withdrawal Amt', 'Deposit Amt', 'Balance'). Tier 2: column names used directly (no position guessing needed). Validator: reconcile rate = 0.997 (1 row fails — a dividend credit with a fractional paise rounding artefact of Rs 0.01, within BALANCE_TOLERANCE). All 340 rows -> Tier 3. LLM calls: 0.

**File 3 —
Phone
photograph
(Canara
Bank,
handwritten
annotation,
85 rows)**

Tier 0: Tesseract OCR. Raw text is noisy. Tier 1: bank name hint 'CANARA' detected. Tier 2: heuristic fails. Reconcile rate = 0.61 (too many OCR misreads). Validator triggers Tier 4. LLM schema discovery call: 1. New schema object cached. Re-parse: reconcile rate rises to 0.91 (still below 0.98 for 7 rows). Tier 5: 7 rows sent to LLM individually. 5 repaired and accepted. 2 still fail (ink smudge, illegible). Final: 83 rows clean, 2 flagged. LLM calls: 1 (Tier 4) + 7 (Tier 5) = 8.

After all three files are processed, the pipeline merges the clean rows from all three into one unified dataframe: $120 + 340 + 83 = 543$ clean rows, 2 flagged, covering three accounts across three banks. Total LLM calls for extraction: 8 (7 Tier-5 row-repair calls + 1 Tier-4 schema-discovery call). The analysis engine receives the 543-row dataframe and runs all 25 detection cases.

11.2 Sample Output Table

Representative rows from the merged clean output:

Date	Time	Narration	Debit	Credit	Balance	Account_ID	Bank_Name	I F S C - C o d e
15/01/2024	10:32	UPI/PhonePe/RAMESH KUMAR	5,000	—	12,000	ACC001	SBI	S B I N 0 0 0 1 2 3 4
22/01/2024	14:15	NEFT/AXISBANK/SUNITA DEVI	—	8,000	20,000	ACC001	SBI	S B I N 0 0 0 1 2 3 4

Date	Time	Narration	Debit	Credit	Balance	Account_ID	Bank_Name	I F S C _ C o d e
03/02/2024	09:00	IMPS/AXIS/TRANSFER OUT	15,000	—	5,000	ACC002	Axis Bank	U T I B 0 0 0 0 1 2 3
03/02/2024	09:47	IMPS/CANARA/RECEIVED	—	14,700	19,700	ACC003	Canara Bank	C N R B 0 0 0 1 4 5 6
10/02/2024	16:30	ATM WDL/SBI ATM MG ROAD	10,000	—	9,700	ACC003	Canara Bank	C N R B 0 0 0 1 4 5 6

Note: the Axis-to-Canara pair (rows 3 and 4) has a Rs 300 difference consistent with a hawala commission — precisely the kind of pattern Detection Case 12 will flag.

12. Output Schema & Storage

12.1 Standard Transaction Schema

Every row accepted by the Validator conforms to this schema, regardless of the source file format or bank:

Field	Type	Description	Source
Date	date	Transaction date (YYYY-MM-DD after normalisation)	Parsed from document

Field	Type	Description	Source
Time	time	Transaction time (HH:MM) if available; null otherwise	Parsed from document or null
Narration	string	Full transaction description / reference text	Parsed from document
Debit	float	Amount debited; 0.0 if credit transaction	Parsed + cleaned (commas stripped)
Credit	float	Amount credited; 0.0 if debit transaction	Parsed + cleaned
Balance	float	Running balance after this transaction	Parsed + validated
Account_ID	string	Internal identifier assigned by the system (ACC001, etc.)	From session upload metadata
Bank_Name	string	Bank name string from Tier 1 metadata	Tier 1 detection
IFSC_Code	string	IFSC code if extractable from document header; null otherwise	Tier 1 detection or null

12.2 Storage Artefacts

Artefact	Description
clean_rows.csv	All rows accepted by the Validator across all uploaded documents. Primary input to the analysis engine.
flagged_rows.csv	All rows that failed the Validator at every tier. Shown to investigator with per-row failure reason.
metadata.json	Metadata receipt: per-document stats (row count, accept rate, schema used, LLM calls, timestamp, any tricky-case flags).
schema_cache/	Directory of cached schema objects keyed by document-format hash. Prevents repeated Tier 4 LLM calls for identical formats.
In-memory dataframe	pandas dataframe loaded from clean_rows.csv at the start of the analysis phase. All 25 detection cases read from this.
ChromaDB collection	Each clean row converted to a descriptive sentence and stored as a vector embedding. Used exclusively by the RAG chatbot.

13. Privacy & Security

Every uploaded bank statement contains sensitive financial data belonging to suspects under active investigation. While the hackathon prototype does not implement the full set of controls below, the design explicitly accounts for them:

- **Encryption at rest:** uploaded statements and generated reports encrypted at rest in production.
- **Encryption in transit:** all frontend-to-backend traffic over HTTPS.
- **Authentication:** investigator login required; only authorised personnel can upload or view reports.
- **Audit logging:** every action (upload, analysis run, report download, chatbot query) logged with investigator ID and timestamp.
- **Data lifecycle:** ChromaDB collections and local files cleared when a case is formally closed.
- **LLM data handling:** only the first 30–40 lines of each document are sent to the LLM at Tier 4. Full transaction data never leaves the local machine except for individual failing rows at Tier 5.
- **Local model path:** switching to a local LLM (Ollama, LM Studio) via the provider-independent interface eliminates all external data transmission entirely.

14. Cost, Performance & Local-Model Path

A common misconception about LLM-assisted extraction is that costs scale with row count. In the Validation-Arbitrated Tiered Hybrid, they do not.

Token Cost Breakdown

Phase	LLM Calls	Tokens Per Call	Scales With
Tier 4 — Schema discovery	0 or 1 per document format	~500–800	Number of distinct document formats (not rows)
Tier 5 — Row repair	0 to N per document	~150–300 per failing row	Number of failing rows (typically small)
Analysis — Thresholds (Case 23)	1 per session	~200–400	Number of sessions
Analysis — Explanations	1 per triggered finding per account	~300–600	Number of flagged accounts x cases triggered
Report — Executive summary	1 per session	~800–1,200	Number of sessions

For a typical hackathon case of 8 suspects with 3 months of statements each (approximately 5,000 rows total), the expected extraction LLM cost is 1–3 Tier 4 calls (most documents share a format) plus a handful of Tier 5 calls for OCR noise. The 5,000 rows themselves are processed by pandas in under one second — no LLM involvement.

The local model path requires no design changes. Replacing the Groq endpoint in `llm_interface.py` with an Ollama endpoint (<http://localhost:11434>) eliminates all API costs and all external data transmission. A 7B-parameter quantised model running on the demo laptop is sufficient for Tier 4 schema discovery on most well-structured documents.

15. Testing & Accuracy Proof

The Anti-Overfitting Law (Section 5) requires correctness to be proven on a blind set — documents from banks not seen during development. This section defines the testing protocol.

Development Test Set

During the mentoring phase, the extraction pipeline is developed and tuned using a synthetic dataset generated by the team's data-generation script. This dataset covers three banks (SBI, HDFC, Axis), four file formats (digital PDF, Excel, CSV, photographed statement), and deliberately plants known parsing challenges (wrapped narrations, missing opening balance, reverse-order statements).

Blind Test Set

Before the hackathon demo, the pipeline is tested against a blind set consisting of: at least two banks not in the development set (e.g., Canara Bank, Union Bank), at least one statement with no running balance column, and at least one phone photograph with deliberate quality degradation (slight blur, rotation). A document passes if its

reconcile rate meets ACCEPT_RECONCILE_RATE on clean rows. Any document that fails triggers a general engine fix, not a bank-specific patch.

Accuracy Metric

The accuracy metric reported to the jury panel is: fraction of planted fraud patterns correctly identified by the 25 detection cases, out of total patterns planted. For extraction specifically: fraction of rows correctly parsed and validated, out of total rows in the blind set. Both metrics are computed automatically by the test script and included in the demo presentation.

16. Unchanged Downstream Phases

The extraction module feeds a unified transaction dataframe to three downstream phases. These phases are unchanged from Version 1.0. They are summarised here for completeness; the full specification is in Sections 5–7 of Version 1.0 (reproduced in Sections 17 onwards of this document).

16.1 Analysis Phase — 25 Detection Cases

The analysis engine receives the clean transaction dataframe together with the investigator's case brief and runs 25 detection cases across seven categories: graph-based analysis (Cases 1–5, NetworkX), time-based analysis (Cases 6–10, pandas), amount-based analysis (Cases 11–15, pandas), counterparty analysis (Cases 16–19, pandas), narration analysis (Cases 20–21, pandas and LLM hybrid), statistical anomaly detection (Case 22, scikit-learn Isolation Forest), and LLM-driven analysis (Cases 23–25). Of the 25 cases, 22 are fully deterministic. The LLM is used only to generate dynamic thresholds (Case 23), apply free-text filter rules from the case brief (Case 24), and generate a fraud-type hypothesis when no case brief is provided (Case 25). Every triggered case produces a structured finding with an LLM-written plain-English explanation, an evidence-strength rating, and a recommended investigator action. A composite suspicion score from 0 to 100 is computed per account from the weighted findings.

16.2 Report Generation

The reporting module consumes the analysis output and produces two artefacts: a court-ready PDF assembled by ReportLab, and a working Excel export assembled by pandas and openpyxl. The PDF is structured for three readers: an executive summary for senior officers (one page, plain language), detailed findings per account for the investigating officer (flagged transactions, score breakdowns, case explanations, money-flow chain), and a technical appendix for forensic experts and lawyers (raw mathematical evidence, NetworkX cycle paths, Isolation Forest scores, balance-validation results). The LLM contributes in exactly two places: the per-finding explanations generated during analysis, and the one-page executive summary. All tables, scores and the technical appendix are produced directly by Python.

16.3 RAG Investigation Chatbot

The RAG chatbot becomes active after report generation. It allows the investigator to ask natural-language questions about the uploaded case data. The pipeline: each clean transaction is converted to a descriptive sentence and stored in ChromaDB as a vector embedding during extraction. When the investigator asks a question, ChromaDB retrieves the 30–50 most semantically relevant transactions, which are passed to the LLM along with the question and conversation history. The LLM generates a grounded answer. Because every answer is derived only from ChromaDB-retrieved transactions, the chatbot cannot invent transactions that do

not exist. The chatbot handles three query categories: filtering and retrieval, calculation and summary, and investigation insight.

17. Requirements

17.1 Functional Requirements

ID	Requirement	Description
FR-01	Multi-format ingestion	PDF (digital and scanned), Excel, CSV, DOCX, JPG/PNG for multiple suspects in one session.
FR-02	Case-brief input	Free-text description of suspected crime pattern, contextualising all analysis steps.
FR-03	OCR processing	Scanned documents and photographs converted to machine-readable text before further processing.
FR-04	Field detection & standardisation	Auto-identify Date, Time, Narration, Debit, Credit, Balance, Account_ID, Bank_Name, IFSC_Code regardless of format.
FR-05	Data cleaning & validation	Duplicates removed, balance consistency verified, flagged rows surfaced for manual review — never silently dropped.
FR-06	Fraud-pattern detection	25 detection cases across graph, time, amount, counterparty, narration, statistical and LLM-driven categories.
FR-07	Round-trip & money-trail	Cyclic money-flow detection (NetworkX) and FIFO-based money-trail tracing (pandas).
FR-08	Money-flow visualisation	Interactive node-edge graph; suspicious accounts, mule chains and convergence points highlighted.
FR-09	Composite suspicion scoring	Score 0–100 per account; per-case point breakdown shown in report.
FR-10	Report generation	Court-ready PDF and Excel export from a single action.
FR-11	RAG chatbot	Natural-language queries answered from actual case data via ChromaDB retrieval.
FR-12	Session isolation	Each case session has its own dataframe, graph, scores and chatbot context.

17.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Process 1,00,000+ transactions spanning multiple suspects within a few minutes. (Satisfied by tiered architecture: pandas handles row volume; LLM calls scale with document count.)
NFR-02	Accuracy & Explainability	All 22 rule-based detection cases fully deterministic and reproducible; every finding traceable to a calculation.
NFR-03	Usability	Usable by an investigator with no technical background; no command-line interaction required.
NFR-04	Reliability	Rows that cannot be extracted or validated with confidence flagged for review, never silently dropped (enforced by Validator).
NFR-05	Scalability	Architecture scales from single-machine demo to multi-office production with configuration changes only.

ID	Category	Requirement
NFR-06	Security & Privacy	Design accounts for encryption, authenticated access, and audit logging (see Section 13).
NFR-07	Maintainability	Extraction, analysis, reporting and chatbot are loosely coupled modules.
NFR-08	Cost Efficiency	LLM calls minimised and cached; extraction costs scale with documents, not rows.

18. System Architecture Overview

18.1 Three Architectural Layers

- **Presentation Layer (React frontend):** upload and case-brief screen, analysis progress screen, results screen with score rankings, interactive network graph, download links, and chatbot panel.
- **Application Layer (Python / FastAPI backend):** extraction pipeline, analysis engine, report generator, and RAG chatbot pipeline. REST endpoints: /upload, /status, /results, /report, /export, /chat.
- **Data Layer:** in-memory pandas dataframe (live working dataset), ChromaDB on disk (chatbot vector store), local file storage (uploads, outputs, cache).

18.2 The Four Processing Phases

- **Phase 1 — Input:** investigator types case brief and uploads files. FastAPI saves files and stores brief in session memory.
- **Phase 2 — Extraction:** every uploaded file converted to one standardised transaction table via the Validation-Arbitrated Tiered Hybrid (this document, Sections 4–12).
- **Phase 3 — Analysis:** 25 detection cases run against standardised table guided by case brief, producing findings and suspicion scores (Section 16.1).
- **Phase 4 — Output:** PDF and Excel reports generated; interactive graph and score rankings sent to frontend; chatbot becomes available (Sections 16.2–16.3).

19. Technology Stack

Category	Technology	Used For
Frontend	React, TailwindCSS, Axios	Web interface, styling, API communication.
Network Graph	Cytoscape.js / D3.js	Interactive money-flow visualisation.
Backend Framework	Python, FastAPI, uvicorn	Core application logic and API layer.
Digital PDF Extraction	pdfplumber	Extracts embedded text from digital PDF statements (Tier 0).
OCR	Tesseract (pytesseract)	Converts scanned documents and photographs to text (Tier 0).
Data Standardisation	pandas	Excel/CSV reading, schema application, validation, analysis.
LLM Interface	llm_interface.py (provider-independent)	Schema discovery (Tier 4), row repair (Tier 5), thresholds, explanations, executive summary. Default provider: Groq. Swappable to local model.
Graph Analysis	NetworkX	Cases 1–5: round-trip, layering, hub, isolated-cluster, convergence.
Statistical Anomaly	scikit-learn (Isolation Forest)	Case 22: self-trained anomaly detection.
Vector Storage	ChromaDB, sentence-transformers	Per-transaction embeddings for RAG chatbot.
Report Generation	ReportLab, openpyxl, matplotlib	PDF and Excel forensic reports, rendered network-graph image.
File Storage	Local disk (uploads/outputs/cache)	Raw uploads, generated reports, cached LLM responses and schema objects.

20. Data Storage & Data Flow

20.1 Storage Components

Component	Technology	Lifetime	Role
In-Memory Dataframe	pandas (RAM)	Session only	Live working table of clean transactions. All 25 detection cases read/write here. Input to NetworkX graph and report generator.
Vector Store	ChromaDB (disk)	Persistent per session	Each transaction as a searchable embedding. Used exclusively by the RAG chatbot.
Local File Storage	Disk: uploads/outputs/cache	Persistent	Raw uploaded files, generated PDF/Excel reports, cached LLM responses, cached schema objects.

20.2 End-to-End Data Flow

- **Step 1:** Investigator types case brief and uploads files -> FastAPI /upload endpoint.
- **Step 2:** FastAPI saves files to uploads folder, stores case brief in session memory, assigns session ID, triggers extraction pipeline.
- **Step 3:** File-type router sends each file to pdfplumber (digital PDF), Tesseract + pattern cleaning (scan/photo), or pandas (Excel/CSV).
- **Step 4:** Tier 2 deterministic parse -> Validator. If pass rate $\geq$ ACCEPT_RECONCILE_RATE: clean rows to Tier 3. Else: trigger Tier 4.
- **Step 5:** Tier 4 LLM schema discovery (one call per format, cached). Re-parse. Re-validate. Failing rows -> Tier 5.
- **Step 6:** Tier 5 LLM row repair (one call per failing row). Validator re-checks. Pass -> Tier 3. Still fail -> flagged_rows.csv.
- **Step 7:** Clean rows written to: (a) in-memory pandas dataframe for analysis, (b) ChromaDB as vector embeddings for chatbot. Metadata receipt saved.
- **Step 8:** LLM reads case brief -> returns dynamic thresholds (Case 23). Applied to all 22 rule-based cases.
- **Step 9:** All 25 detection cases run: NetworkX (Cases 1-5), pandas (Cases 6-21), scikit-learn (Case 22), LLM (Cases 23-25).
- **Step 10:** Every triggered finding sent to LLM -> plain-English explanation, evidence-strength rating, recommended action.
- **Step 11:** Composite suspicion scores computed per account. Accounts ranked.
- **Step 12:** LLM writes executive summary. ReportLab generates PDF. openpyxl generates Excel. Both saved to outputs folder.
- **Step 13:** FastAPI sends findings, scores, graph data to React frontend. Interactive network graph rendered. Score rankings and download links shown.
- **Step 14:** Chatbot panel active. Each investigator question -> ChromaDB semantic retrieval -> LLM generates grounded answer.
- **Step 15:** Investigator downloads PDF report and submits to case file.

21. Security & Privacy Considerations

Security and privacy are covered in full in Section 13. The key production controls are: encryption at rest and in transit, investigator authentication, audit logging, data lifecycle management, and the option to run the local model path to eliminate all external data transmission.

22. Scalability & Deployment Considerations

The hackathon prototype runs on a single machine. The architecture scales to production deployment across CID Karnataka's offices with configuration changes only:

- **Backend deployment:** FastAPI deployed on a cloud server or container cluster for concurrent multi-investigator access.
- **Session isolation:** each case session already has its own dataframe and ChromaDB collection; concurrent sessions do not interfere.
- **Storage migration:** local folders replaced by AWS S3 or Google Cloud Storage; local ChromaDB replaced by a managed vector database.
- **No core logic changes:** extraction, analysis, reporting and chatbot logic remain identical between hackathon demo and production rollout.

23. Implementation Roadmap

23.1 Recommended Build Order

- **1.** Build Tier 2 deterministic engine and Validator (Sections 8 and 7) assuming clean CSV input. Validate on hand-crafted test data.
- **2.** Add Tier 0 pdfplumber path (digital PDFs). Feed into existing engine and Validator.
- **3.** Add Tier 4 LLM schema discovery. Test against multiple bank formats. Verify caching.
- **4.** Add Tier 0 Tesseract OCR path (scans and photos). Most complex — tackled after engine is proven.
- **5.** Add Tier 5 LLM row repair. Test on deliberately degraded OCR outputs.
- **6.** Add Tier 1 metadata extraction.
- **7.** Wire ChromaDB ingestion from Tier 3 clean rows.
- **8.** Build 25 detection cases: deterministic pandas cases first, then NetworkX, then scikit-learn, then LLM-driven cases (23–25).
- **9.** Build report generator (Excel first, then PDF section by section).
- **10.** Build RAG chatbot pipeline and conversation-history handling.
- **11.** Build React frontend and wire to FastAPI endpoints for end-to-end integration.
- **12.** Run blind-set accuracy test. Fix any general engine issues found.

23.2 Hackathon Timeline

- **Mentoring Phase (05–25 June 2026):** Finalise this design document; build and validate Tiers 0–5 against synthetic dataset; build core deterministic detection cases.
- **Hackathon Days (4–5 July 2026):** Complete LLM-driven cases; build report generator and RAG chatbot; build frontend; integrate end-to-end; prepare demo.

24. Team Structure & Roles

Member	Primary Role	Responsibilities
Member 1	AI & Data Ingestion	Multi-format parsing, OCR integration, Tiers 0–5 extraction pipeline, LLM interface module, ChromaDB embedding pipeline.
Member 2	Fraud Detection	Amount-based, time-based and counterparty-based detection cases (Categories B, C, D), hawala/smurfing/round-tripping logic, Isolation Forest (Category F).
Member 3	Graph & Visualisation	Transaction graph construction, five graph-based detection cases (Category A), interactive fund-flow network visualisation (Cytoscape.js / D3.js).
Member 4	RAG + Frontend	ChromaDB RAG pipeline and chatbot (Section 16.3), React frontend screens, forensic PDF/Excel report generator (Section 16.2).

25. Conclusion

Version 2.0 of this System Design Report documents the corrected and implemented architecture for the Multi-Accused Cross-Account Investigation Engine. The central change from Version 1.0 is the replacement of the original extraction design with the Validation-Arbitrated Tiered Hybrid: a six-tier pipeline in which a deterministic validator acts as the sole arbiter of whether a parsed row is accepted, escalated, or flagged.

The new architecture resolves all four problems identified with the original design: the contradictory parsing strategies are replaced by one deterministic engine with a runtime-discovered schema; token costs scale with documents rather than rows; the Anti-Overfitting Law prevents bank-specific hard-coding; and the Validator's explicit flag-and-surface behaviour eliminates the silent data-loss bug.

The downstream phases — the 25 detection cases, the report generator, and the RAG chatbot — are unchanged from Version 1.0 and continue to embody the system's core design principle: every numeric finding is produced by deterministic code, and the LLM is reserved for tasks where it provides genuine value over code. Combined with the Validation-Arbitrated Tiered Hybrid extraction layer, this gives the system the accuracy, reproducibility, and bank-agnostic generality required for forensic use by CID Karnataka investigators.

With this Version 2.0 design on record, the team proceeds to complete the remaining implementation phases ahead of the CIDECode Hackathon 2026 on 4–5 July 2026.

Document version: 2.0 — June 2026 | **Supersedes:** System Design Report Version 1.0 | **Team:** Survey Corps | **Event:** CIDECode Hackathon 2026, CCITR – CID Karnataka / PES University